

Services and Routing in Angular

Angular's architecture relies on two key pillars: Services and Routing. These concepts form the backbone of scalable, maintainable single-page applications.

Let's explore how they work together to create powerful web experiences.

 **by Saratha Natarajan**



What Are Angular Services?



Singleton Objects

Services are single instances that persist throughout your application's lifecycle.



Shared Resources

They provide data, logic, or state sharing across components.



Dependency Injection

Angular's DI system makes services available where needed.

Angular Flow



Baala

@Bsreclon



```

yeh 1 ##[injectable"<del>de</del>er<del>a</del> >
ope 9   injectable']
ltd 9   derse<del>ute</del>' #[lectable]>
ete 1   #<del>3</del> degeententer de<del>ject</del>our"]>
=> 5   drepentey inje[]
ean 9   derpenuter injecthbr"]>
elo X   denpcudancy within :ell"evendennderingen compoructor">
llo 2   >
ex6 0   >

```

Creating and Injecting Services

Create the Service

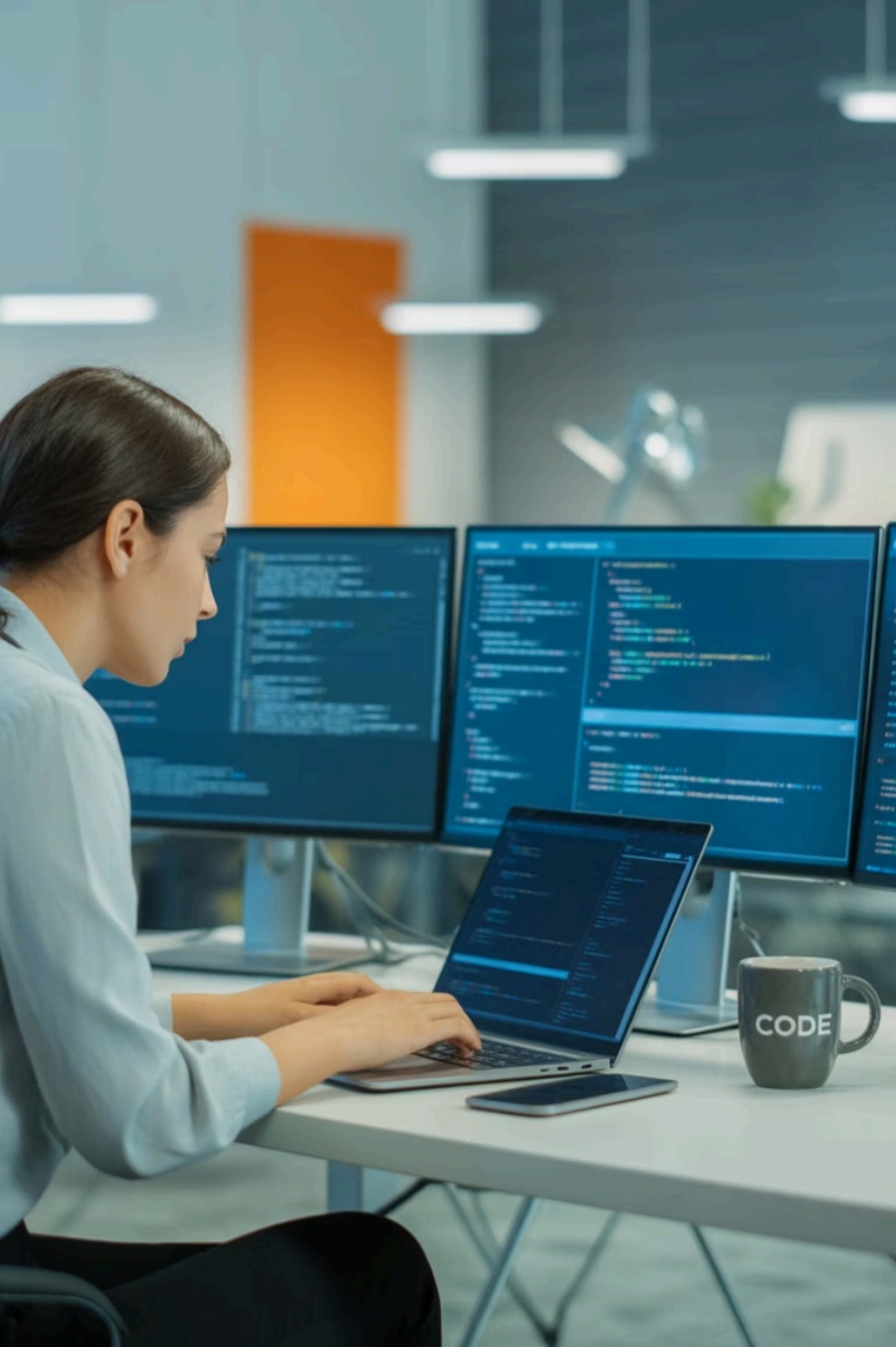
Use the `@Injectable()` decorator to mark a class as a service.

Register the Provider

Provide in root for app-wide access or in feature modules for limited scope.

Inject the Service

Add the service to component constructors to make it available.



Practical Service Examples

Data Services

Use HttpClient to fetch data from APIs. Abstract API calls behind intuitive methods.

Authentication

Manage user login state, tokens, and session information. Protect sensitive routes.

Utility Services

Handle logging, error tracking, and application-wide notifications.

What Is Routing in Angular?

Navigation Framework

Angular Router enables movement between different application views without page reloads.

It's essential for creating smooth, app-like experiences on the web.

Core SPA Feature

Routing is what makes single-page applications feel like traditional multi-page websites.

Users can bookmark specific states and use browser history normally.



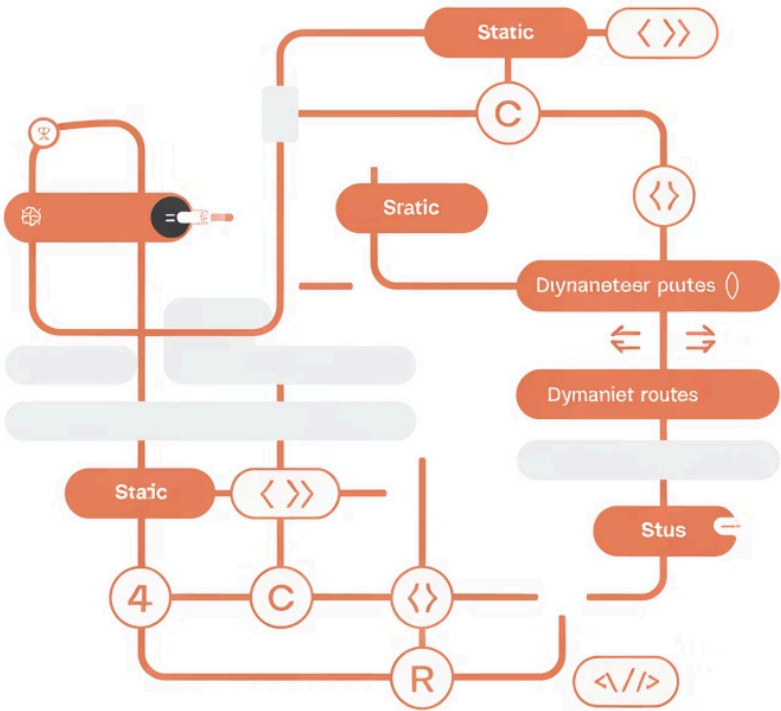
Three icons arranged vertically. The top icon is a code symbol (</>). The middle icon is a fountain pen. The bottom icon is a calendar or document with a grid.

Define an array of route objects mapping paths to components.

Use `RouterModule.forRoot(routes)` in your `AppModule`.

Place `<router-outlet>` in your template where views should appear.

Angular Routing



Defining Routes: Paths and Components

- ## 1 Static Routes

Simple paths like 'dashboard' or 'products' that map directly to components.
- ## 2 Dynamic Routes

Parameterized paths like 'product/:id' capture values from the URL.
- ## 3 Wildcard Routes

Special '**' path catches any undefined routes for 404 handling.
- ## 4 Redirect Routes

Routes with redirectTo property send users to another path.

```

ta roconsalal ligxial sniet bnsnliie "tcanote 'llilgois' i, cynkigiae "etiarozto" l>
ee | rouliail' leketetce
ze | ongvlaro "ilructs" >
zu =>
no | routeae' ]">
ze | | router"lljeitfiliaaanc liee ryuter">
ne | | | restesic rutto">
a | | | rosaustit rytyoia" >
zu | | | anestul tal"="artes" ]>
ia | | | restertit "iliiol"="routes">

```

Route Configuration Example

```

const routes: Routes = [
  { path: 'dashboard', component: DashboardComponent },
  {
    path: 'products',
    component: ProductListComponent,
    children: [
      { path: 'top', component: TopProductsComponent },
      { path: ':id', component: ProductDetailComponent }
    ]
  },
  { path: '', redirectTo: '/dashboard', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent }
];

```


Navigating Programmatically

Template Links

Use routerLink directive on anchor tags for declarative navigation.

History Control

Use navigateByUrl() for absolute URLs or navigate with relativeTo option.



Router Service

Inject Router and use navigate() method for programmatic control.

Pass Parameters

Include route params, query params, and fragments as needed.

Route Guards and Lazy Loading

Route Guards

Guards control access to routes based on conditions:

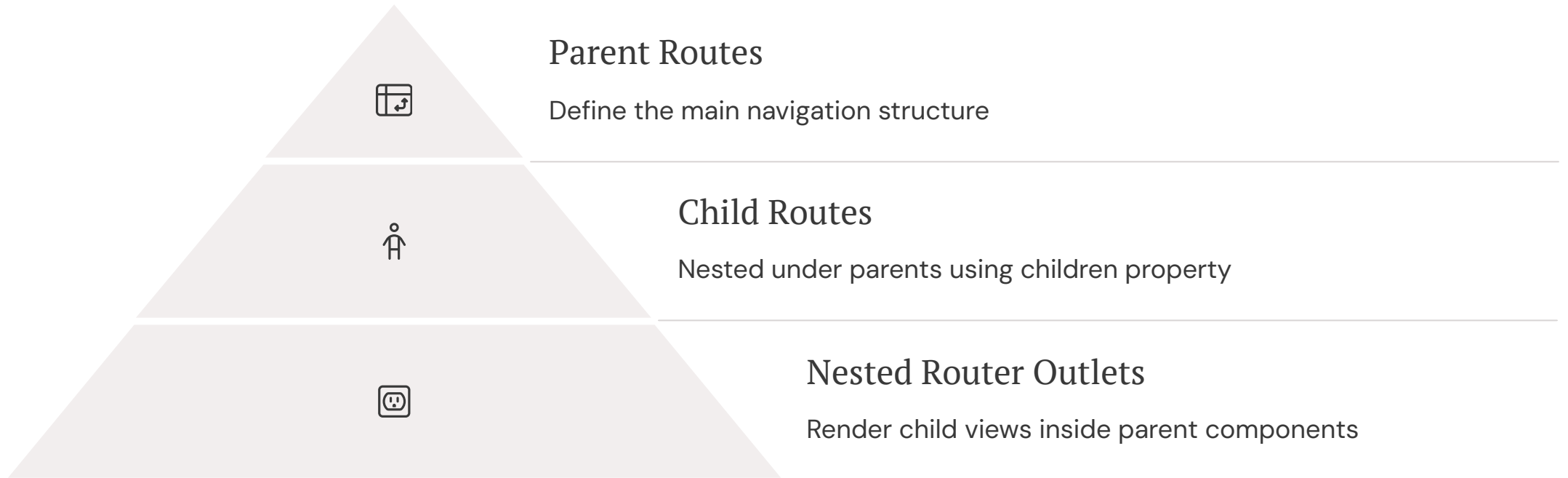
- CanActivate: permission to enter
- CanDeactivate: permission to leave
- Resolve: pre-fetch data

Lazy Loading

Load feature modules on demand:

- Reduces initial bundle size
- Improves startup time
- Uses loadChildren syntax

Nested and Child Routes



Hierarchical routing creates complex UIs like dashboards with tabs or admin panels with sub-sections. Each level can have its own navigation.

Angular Location Strategies



PathLocationStrategy

Default strategy using HTML5 History API



HashLocationStrategy

Uses hash fragment for compatibility

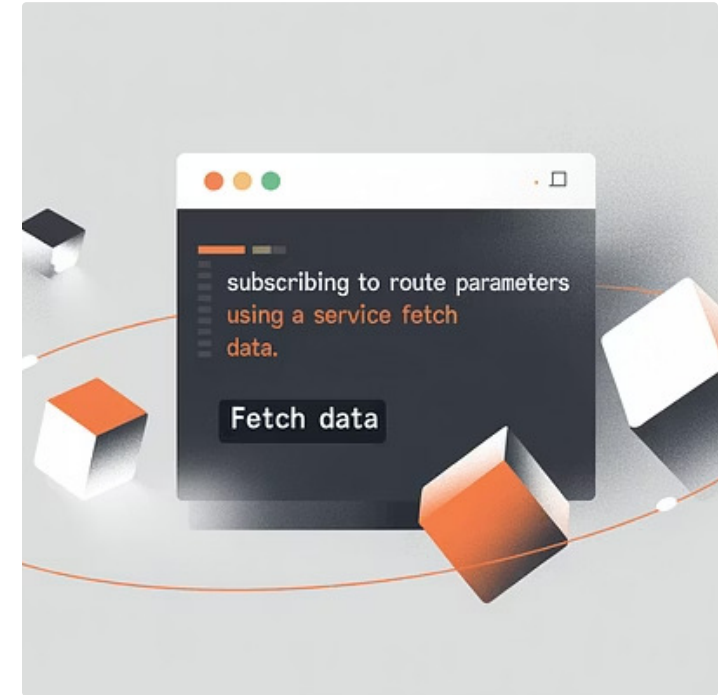
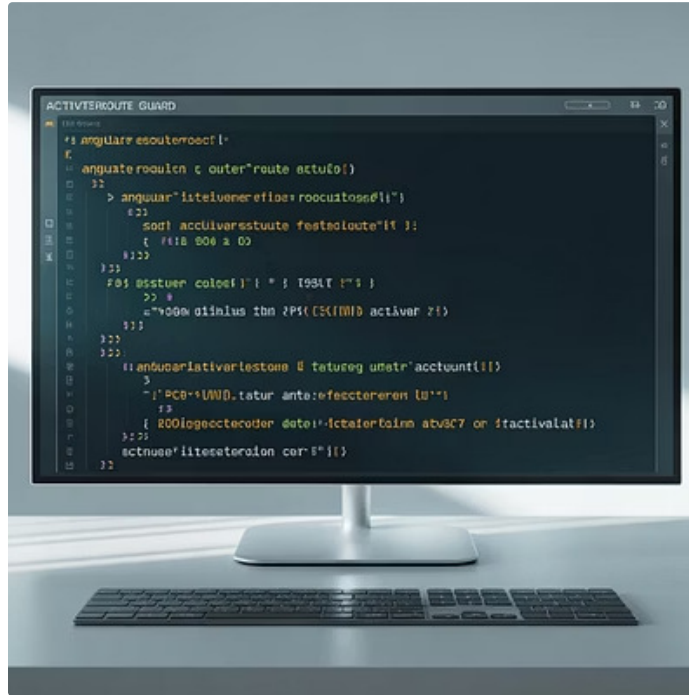
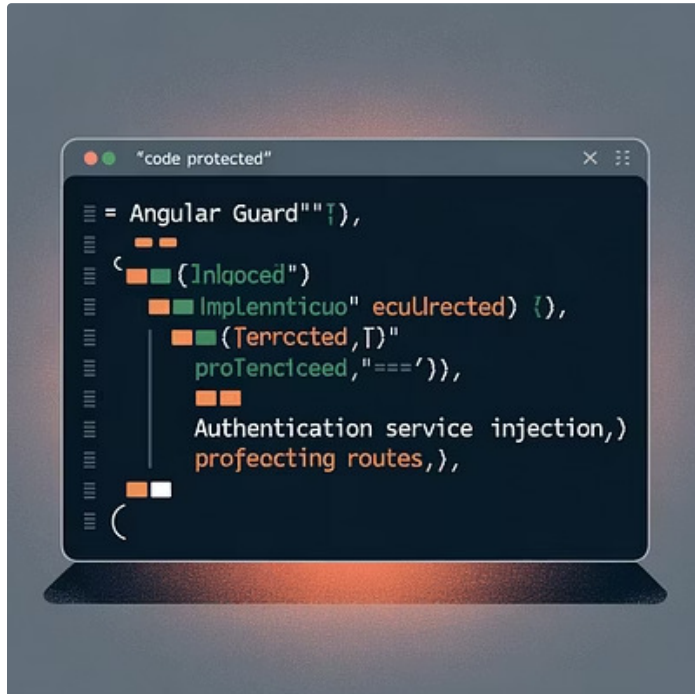


Server Configuration

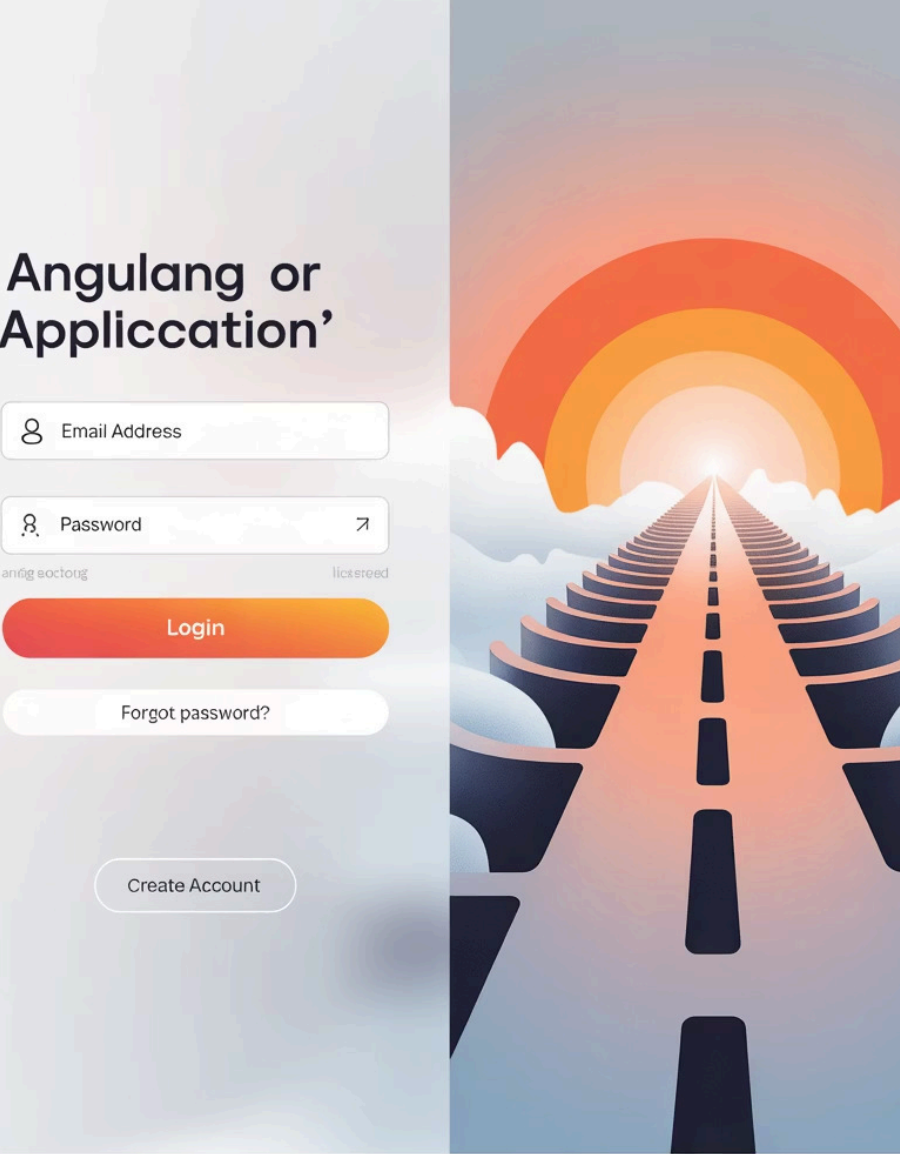
Path strategy requires server-side URL rewriting

The default PathLocationStrategy creates clean URLs like `example.com/products`. HashLocationStrategy uses fragments like `example.com/#/products` for legacy support.

Services and Routing Together



Services and routing work together to create dynamic applications. Services fetch data based on route parameters or authenticate users before allowing navigation.



Common Routing Scenarios

404

Not Found Handling

Capture unknown routes and redirect to custom error page.

Auth

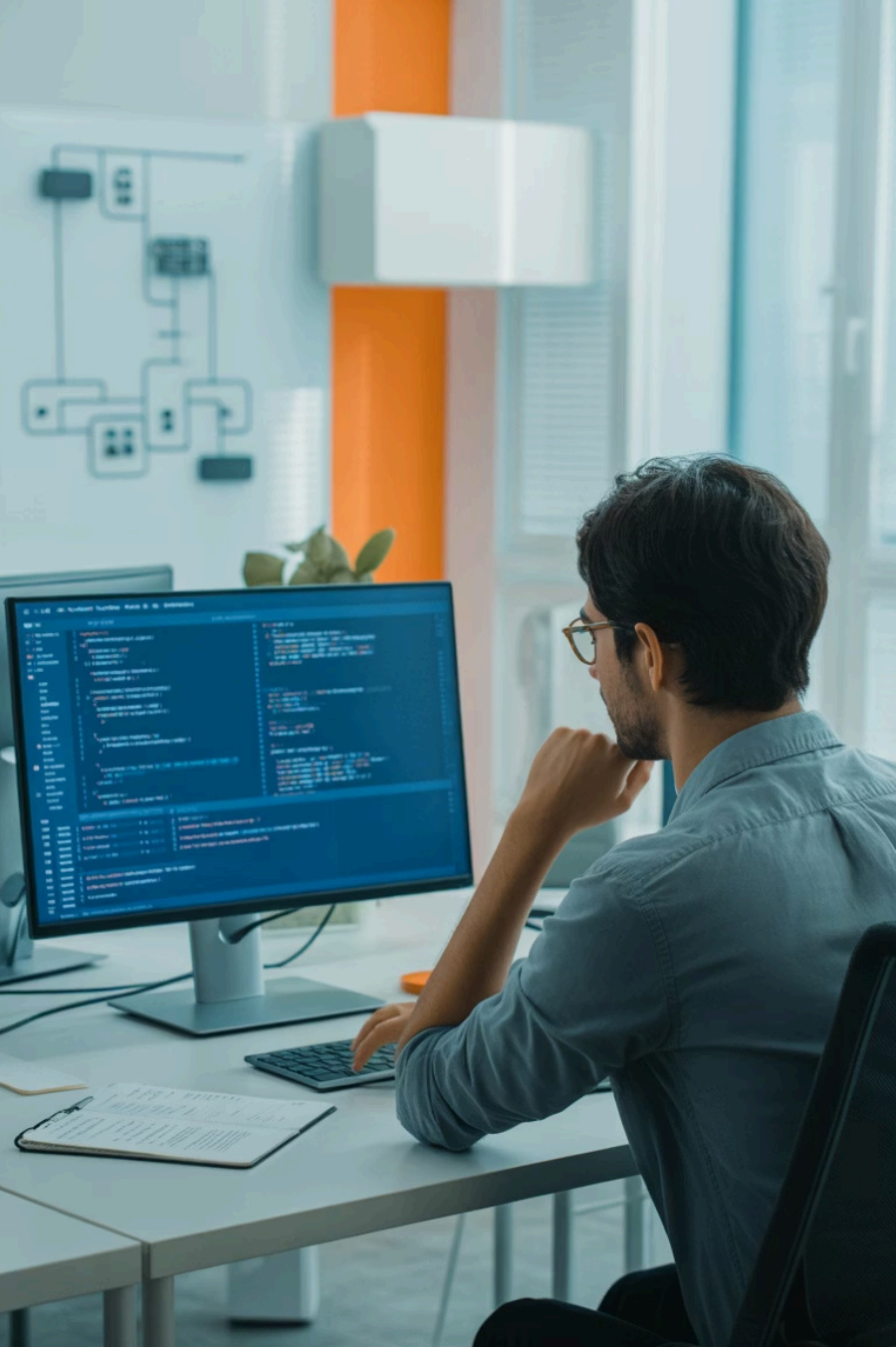
Authentication

Redirect unauthorized users to login page, return to intended destination after.

Data

Parameter Extraction

Get route params, query params, and pass to services.



Summary and Best Practices



Service Best Practices

Keep services focused on specific concerns. Use proper injection scope. Test thoroughly.



Routing Best Practices

Organize routes logically. Use lazy loading for large apps. Implement proper guards.



Integration Patterns

Let services respond to route changes. Pre-load data with resolvers. Share state across views.